



Sou9i

Application mobile de **caisse intelligente** & gestion pour commerces de proximité

Une solution SaaS mobile qui remplace la caisse enregistreuse classique : vente au comptoir, vente au poids, gestion de stock en temps réel, facturation et tableau de bord analytique — conçue spécifiquement pour le marché tunisien.

Flutter · Firebase

Android · iOS

SaaS · Freemium

Point of Sale (POS)

Marché tunisien

TYPE DE PROJET

Application mobile (POS)

SECTEUR CIBLE

Épicerie · Torréfaction · Fruits secs

DURÉE ESTIMÉE

1 à 2 mois (stage)

STACK TECHNIQUE

Flutter + Firebase + SQLite

Table des matières

01	Contexte & Problématique	03
02	La Solution — Sou9i	04
03	Analyse Concurrentielle	05
04	Cahier des Charges Fonctionnel	06
05	Fonctionnalités MVP	07
06	Fonctionnalités à Forte Valeur Ajoutée	08
07	Modèle Économique & Marché Tunisien	09
08	Architecture Technique Détaillée	10
09	Sécurité de l'Application	11
10	Base de Données — Modèle Relationnel	12
11	Diagramme de Cas d'Utilisation (UML)	13
12	Diagramme de Classes (UML)	14
13	Diagramme de Séquence (UML)	15
14	Diagramme d'Activités (UML)	16
15	Cas de Tests Fonctionnels	17
16	Budget & Faisabilité Technique	18
17	Risques du Projet & Mitigation	19
18	Planning du Projet	20
19	Maquettes de l'Application	21
20	Conclusion & Perspectives	22

Un marché sous-digitalisé

En Tunisie, les commerces de proximité — épiceries, magasins de fruits secs (*hmas*), herboristeries, torréfacteurs — représentent une part considérable du tissu économique local. Pourtant, leur gestion reste largement manuelle ou s'appuie sur des logiciels de caisse conçus pour des PC de bureau, inadaptés à leurs contraintes réelles.

△ Limites des solutions actuelles

- **Coût prohibitif** : l'acquisition d'un PC, d'un logiciel sous licence et du matériel périphérique (scanner, imprimante) représente un investissement difficile à amortir pour un petit commerce.
- **Immobilité** : la caisse est fixée à un poste unique. Toute consultation à distance est impossible, ce qui prive le propriétaire d'une vision en temps réel de son activité.
- **Gestion de stock défaillante** : les erreurs de comptage, les ruptures non anticipées et les pertes liées aux dates de péremption sont récurrentes.
- **Vente au poids ignorée** : les fruits secs, graines et épices se commercialisent à la masse — une spécificité que la plupart des outils généralistes gèrent mal ou pas du tout.
- **Absence de pilotage analytique** : aucun tableau de bord, aucune statistique de vente, aucune aide à la décision d'achat pour le propriétaire.



Coût élevé

PC + licence + matériel : investissement lourd, ROI incertain pour un petit commerce.



Perte de temps

Comptage manuel, calcul des prix au poids, recherche des recettes — tout est fastidieux.



Zéro visibilité

Aucune statistique fiable pour anticiper les réapprovisionnements ou analyser la rentabilité.

Notre proposition : Sou9i

Sou9i est une application mobile multiplateforme (Android & iOS) qui transforme un simple smartphone ou une tablette en caisse enregistreuse professionnelle et en outil de gestion intégré. Conçue pour les commerces de proximité tunisiens, elle offre toutes les fonctions d'un logiciel de caisse haut de gamme, sans PC, sans licence onéreuse.

✓ Proposition de valeur

Toute la puissance d'un logiciel de caisse professionnel — vente, stock, facturation, analytique — dans la poche du commerçant, à une fraction du coût, avec une prise en main immédiate et une gestion native de la vente au poids.

Deux profils utilisateurs

Caissier

Effectue les ventes (comptoir et au poids), scanne les produits, encaisse et imprime les tickets. Accès limité à son périmètre.

Administrateur (patron)

Supervise l'ensemble de l'activité : gestion des produits, du stock, des dépenses, des statistiques et des rapports — en temps réel, depuis n'importe où.

Parcours type d'une vente

- **Scanner** le code-barres ou sélectionner le produit dans le catalogue.
- **Saisir le poids** si le produit est vendu au kilogramme — calcul automatique du montant.
- **Valider le panier** et choisir le mode de paiement (espèces, carte, D17...).
- **Imprimer** le ticket via imprimante thermique Bluetooth ou générer une facture PDF.
- Le stock est **décrémenté automatiquement** et les statistiques mises à jour en temps réel.

Positionnement sur le marché

L'analyse des solutions existantes révèle une opportunité claire : aucun acteur ne cible spécifiquement les commerces de proximité tunisiens avec une offre mobile adaptée à la vente au poids.

Solution	Type	Vente au poids	Mobile natif	Marché TN	Prix mensuel	Hors-ligne
Logiciels PC locaux	Desktop	Partiel	✗	Oui	Licence unique	✓
Square POS	Mobile / Web	✗	✓	✗	~\$60/mois	Limité
SumUp	Mobile	✗	✓	Partiel	Gratuit + frais %	✗
Lightspeed	Cloud	✗	✓	✗	~\$100/mois	Limité
Solutions ERP locales	Desktop / Web	Partiel	✗	Oui	Abonnement élevé	✓
✦ Sou9i	Mobile natif	✓ Natif	✓	✓ Conçu pour TN	~30 DT/mois	✓

♥ Avantages différenciants

- Seule solution mobile avec vente au poids intégrée nativement
- Conçue pour et par le marché tunisien (DT, TVA, timbre, D17)
- Tarification accessible — modèle freemium, pas de matériel requis
- Interface bilingue arabe/français avec support RTL

🎯 Cible principale

- Magasins de fruits secs, épices, herboristeries
- Épiceries et boulangeries de quartier
- Commerces informels en voie de formalisation
- Chaînes de commerces de proximité (multi-magasins)

Spécifications **par rôle**



Caissier — Fonctions autorisées

Gestion des ventes

- Rechercher et sélectionner un produit (nom, code-barres)
- Scanner un code-barres via la caméra
- Effectuer une vente au comptoir ou au poids
- Gérer le panier (ajout, suppression, modification de quantité)
- Valider une vente et sélectionner le mode de paiement
- Imprimer ou partager un ticket / une facture
- Enregistrer une vente à crédit (karné client)

Consultation limitée

- Consulter le catalogue et les prix
- Visualiser l'historique de ses propres ventes



Administrateur — Accès complet

Gestion du catalogue

- Créer, modifier, supprimer des produits et catégories
- Définir le prix de vente et le prix d'achat (marge)
- Configurer la vente au poids et les seuils d'alerte
- Générer et imprimer des codes-barres

Gestion opérationnelle

- Gérer le stock et les entrées fournisseurs
- Consulter et valider le crédit client (karné)
- Enregistrer les dépenses (loyer, charges, achats)
- Clôturer la caisse en fin de journée

Pilotage & reporting

- Tableau de bord : recettes, bénéfice, tickets, top produits
- Statistiques par période (jour / semaine / mois)
- Exporter des rapports PDF
- Gérer les comptes utilisateurs et les rôles



Exigences non fonctionnelles

Performance : chargement des listes < 2s ; traitement d'une vente < 1s.

Disponibilité : fonctionnement complet hors-ligne (mode SQLite local).

Sécurité : authentification JWT, chiffrement des données sensibles, RBAC strict.

Périmètre du **premier livrable**

Ces fonctionnalités constituent le cœur du produit livrable à l'issue du stage. Elles ont été sélectionnées selon leur impact opérationnel immédiat et leur faisabilité dans le délai imparti.

CATALOGUE

Gestion des produits

Création, modification et suppression de produits : nom, photo, prix de vente, prix d'achat, catégorie, code-barres. Recherche instantanée par nom ou par scan caméra.

VENTE

Mode caisse (Point of Sale)

Sélection ou scan des articles → panier → calcul automatique du total → encaissement multi-modes (espèces, carte) → génération de ticket ou facture.

SPÉCIFIQUE HMAS

Vente au poids

Saisie du poids sur pavé numérique dédié. Calcul automatique : $\text{montant} = \text{prix/kg} \times \text{poids saisi}$. Indispensable pour les fruits secs, graines et épices.

STOCK

Gestion de stock

Suivi des quantités en temps réel, décrémentation automatique à chaque vente, alerte configurable de stock faible avant rupture.

FACTURATION

Factures & tickets

Génération et archivage des factures (PDF). Impression via imprimante thermique Bluetooth (protocole ESC/POS). Historique consultable et filtrable.

PILOTAGE

Tableau de bord Admin

KPIs en temps réel : recettes, bénéfice, nombre de tickets. Graphiques des ventes hebdomadaires et classement des produits les plus performants.

Authentification & Contrôle d'accès

Connexion sécurisée avec deux niveaux d'accès : **Administrateur** (configuration et supervision complètes) et **Caissier** (opérations de vente uniquement). Chaque compte est individuel et associé à un magasin.

Fonctionnalités à forte valeur

Ces fonctionnalités transforment Sou9i d'un simple outil de caisse en un véritable système de gestion métier, et constituent les arguments de différenciation les plus forts face à la concurrence.

FIDÉLITÉ CLIENT

Crédit client (« karné »)

Enregistrement des ventes à crédit : solde dû par client, historique des achats, envoi de rappels. Une pratique culturelle incontournable en Tunisie, absente des solutions généralistes.

RENTABILITÉ

Calcul de marge & bénéfice réel

Grâce au prix d'achat renseigné, l'application calcule le bénéfice net par article et par période — pas seulement le chiffre d'affaires. Le propriétaire sait précisément ce qu'il gagne.

TRÉSORERIE

Gestion des dépenses

Saisie des charges fixes et variables (loyer, électricité, paiements fournisseurs) pour obtenir une vision complète du cash flow opérationnel.

ANTI-GASPILLAGE

Alertes de péremption

Notification proactive avant qu'un produit n'arrive à son terme. Réduit significativement les pertes sur les denrées alimentaires et les produits à durée de vie limitée.

FIN DE JOURNÉE

Clôture de caisse

Rapport journalier automatique : total des ventes, montant espèces attendu, détection des écarts. Facilite la comptabilité et la supervision à distance.

INTELLIGENCE

Réapprovisionnement intelligent

Analyse de l'historique des ventes pour suggérer automatiquement les produits à commander et les quantités optimales — première brique d'intelligence artificielle.



Sauvegarde cloud & mode hors-ligne

Synchronisation automatique avec Firebase Cloud : aucune donnée perdue en cas de perte ou de changement de téléphone. L'application reste pleinement opérationnelle sans connexion Internet grâce à une base SQLite locale, avec réconciliation transparente dès la reconnexion.

Modèle économique

Sou9i est conçu selon un modèle SaaS (Software as a Service) avec une offre freemium permettant d'acquérir des utilisateurs rapidement et de les convertir vers des abonnements payants.

GRATUIT

Découverte

Caisse de base, 1 utilisateur, catalogue limité à 50 produits. Pour tester sans risque.

0 DT

★ POPULAIRE

Pro

Stock complet, factures, statistiques, crédit client, 5 utilisateurs, multi-catégories.

≈ 30 DT / mois

ENTREPRISE

Multi-magasins

Plusieurs points de vente, dashboard web consolidé, API, support dédié.

Sur devis

Revenus récurrents (SaaS)

Un abonnement mensuel génère des revenus stables et prévisibles. Bien plus rentable et scalable qu'une vente unique de logiciel.

Modèle Freemium

La version gratuite maximise l'acquisition. La conversion vers les offres payantes s'effectue naturellement avec la croissance du commerce.

Spécificités du marché tunisien

Bilingue AR/FR

Interface complète en arabe et en français, avec support natif de l'écriture de droite à gauche (RTL) et du clavier arabe.

Facture conforme

Factures incluant TVA (taux tunisiens), timbre fiscal et mentions légales — conformes à la réglementation fiscale locale.

Paiement mobile

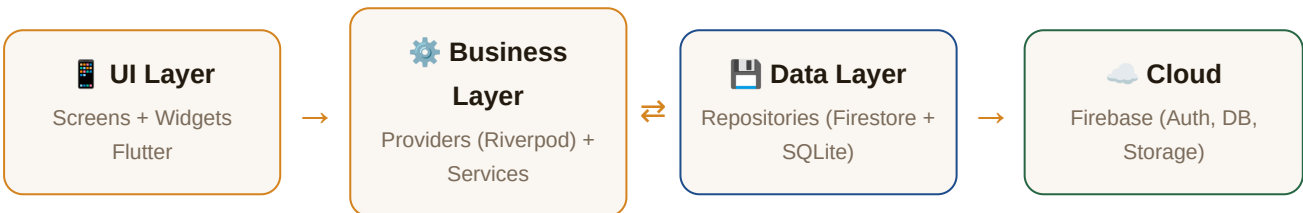
Intégration prévue avec D17, Flouci et e-DINAR pour accepter les paiements électroniques mobiles.

Architecture technique détaillée

Stack technologique

Couche	Technologie retenue	Justification	Alternative
Frontend mobile	Flutter (Dart)	Code unique Android/iOS, UI 60fps, écosystème riche	React Native
Backend / API	Firebase	Temps réel, auth intégrée, déploiement rapide	Node.js + Express
Base de données cloud	Cloud Firestore	NoSQL temps réel, synchronisation offline native	MySQL / PostgreSQL
Base locale (offline)	SQLite (sqflite)	Léger, performant, intégré dans Flutter	Hive / Isar
Authentification	Firebase Auth + JWT	Sécurisé, gestion des sessions et rôles	Supabase Auth
Scan code-barres	mobile_scanner	Rapide, supporte EAN-8/13, QR code	Scanner Bluetooth
Graphiques	fl_chart	Performant, animations, barres / courbes / camembert	syncfusion_charts
Impression tickets	blue_thermal_printer	ESC/POS Bluetooth, compatible imprimantes 58mm/80mm	Print PDF
State management	Riverpod	Réactif, testable, découplage UI/logique	BLoC / Provider
Outils & qualité	Git/GitHub · Figma · Postman	Versioning, maquettes, test API REST	—

Architecture en couches



Les couches UI et métier ne communiquent **jamais directement** avec Firestore. Tout transit par des *Repository classes* qui abstraient la source (cloud ou local), permettant une migration de backend sans toucher à l'interface utilisateur, et facilitant les tests unitaires par injection de dépendances.

Sécurité de l'application

Authentification & Sessions

- **Firestore Auth** : authentification par email/mot de passe avec tokens JWT signés RS256.
- **Refresh tokens** gérés automatiquement, sessions expirantes après inactivité configurable.
- **Hashage des mots de passe** côté Firebase (bcrypt) — jamais stockés en clair.
- **Verrouillage automatique** après N tentatives échouées.

Contrôle d'accès (RBAC)

- **Rôles définis** : *admin* et *caissier* — stockés dans Firestore et vérifiés côté serveur.
- **Firestore Security Rules** : chaque règle vérifie le token JWT et le rôle avant toute opération.
- Le caissier ne peut **jamais** accéder aux données admin, même en manipulant l'URL.

Protection des données

- **TLS/HTTPS** sur toutes les communications réseau (imposé par Firebase).
- **Chiffrement local** : les données SQLite sensibles sont chiffrées avec SQLCipher.
- **Isolation par magasin** : chaque document Firestore contient un *storeId* — un utilisateur ne peut accéder qu'aux données de son propre magasin.
- **Pas de logs de données sensibles** (prix d'achat, marges) en mode debug.

Sauvegarde & Reprise

- **Sauvegarde cloud automatique** quotidienne via Firebase Backup.
- **Offline-first** : SQLite local assure la continuité sans connexion. La synchro reprend automatiquement à la reconnexion.
- **Versionnage des données** : horodatage de chaque modification pour faciliter la réconciliation.
- **Export PDF/CSV** des données critiques à la demande de l'administrateur.

Règle Firestore — Exemple

```
match /ventes/{venteId} {
  allow read: if request.auth != null
    && request.auth.token.storeId == resource.data.storeId;
  allow write: if request.auth != null
    && request.auth.token.role in ['admin', 'caissier'];
}
```

Base de données relationnelle

Modèle logique des entités principales. Firestore étant NoSQL, ce schéma représente la structure logique utilisée côté SQLite (offline) et reflète les collections Firestore.

utilisateurs PK: id		
 id	<i>VARCHAR(36)</i>	UUID (Firebase UID)
nom	<i>VARCHAR(100)</i>	Nom complet
email	<i>VARCHAR(150)</i>	Identifiant unique
role	<i>ENUM</i>	'admin' 'caissier'
 magasin_id	<i>VARCHAR(36)</i>	FK → magasins.id

ventes PK: id		
 id	<i>VARCHAR(36)</i>	UUID
date_heure	<i>TIMESTAMP</i>	Date et heure de vente
total	<i>DECIMAL(10,3)</i>	Montant total TTC
mode_paiement	<i>ENUM</i>	'especes' 'carte' 'credit'
 caissier_id	<i>VARCHAR(36)</i>	FK → utilisateurs.id
 client_id	<i>VARCHAR(36)</i>	FK → clients.id (nullable)

produits PK: id		
 id	<i>VARCHAR(36)</i>	UUID
nom	<i>VARCHAR(200)</i>	Nom du produit
prix_vente	<i>DECIMAL(10,3)</i>	Prix TTC en DT
prix_achat	<i>DECIMAL(10,3)</i>	Pour calcul marge
code_barres	<i>VARCHAR(30)</i>	EAN-8/13 (nullable)
vendu_au_poids	<i>BOOLEAN</i>	Vente kg ou pièce
 categorie_id	<i>VARCHAR(36)</i>	FK → categories.id
 fournisseur_id	<i>VARCHAR(36)</i>	FK → fournisseurs.id

lignes_vente PK: id		
 id	<i>VARCHAR(36)</i>	UUID
 vente_id	<i>VARCHAR(36)</i>	FK → ventes.id
 produit_id	<i>VARCHAR(36)</i>	FK → produits.id
quantite	<i>DECIMAL(10,3)</i>	Nombre de pièces
poids_kg	<i>DECIMAL(10,3)</i>	Poids si vente au poids
prix_unitaire	<i>DECIMAL(10,3)</i>	Prix au moment de la vente
sous_total	<i>DECIMAL(10,3)</i>	= prix × (qté ou poids)

stock PK: id		
 id	<i>VARCHAR(36)</i>	UUID
 produit_id	<i>VARCHAR(36)</i>	FK → produits.id (1-1)
quantite	<i>DECIMAL(10,3)</i>	Stock actuel (kg/pcs)
seuil_alerte	<i>DECIMAL(10,3)</i>	Seuil de déclenchement

clients PK: id		
 id	<i>VARCHAR(36)</i>	UUID
nom	<i>VARCHAR(100)</i>	Nom du client
telephone	<i>VARCHAR(20)</i>	Pour rappels crédit
credit_total	<i>DECIMAL(10,3)</i>	Solde dû cumulé

Diagramme de cas d'utilisation

Ce diagramme identifie les acteurs et les fonctionnalités auxquelles ils ont accès. Le **Caissier** effectue les opérations de vente ; l'**Administrateur** hérite de tous ses droits et supervise en plus la gestion globale.

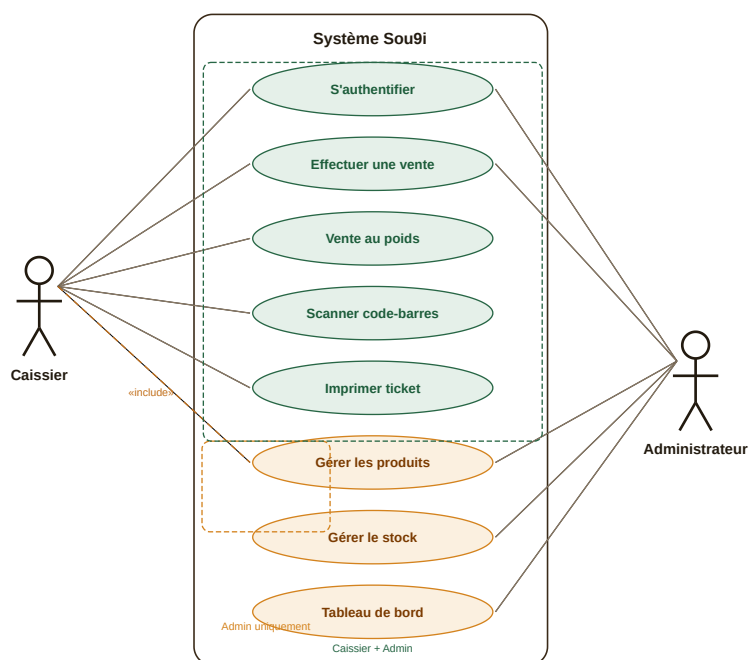


Diagramme de classes

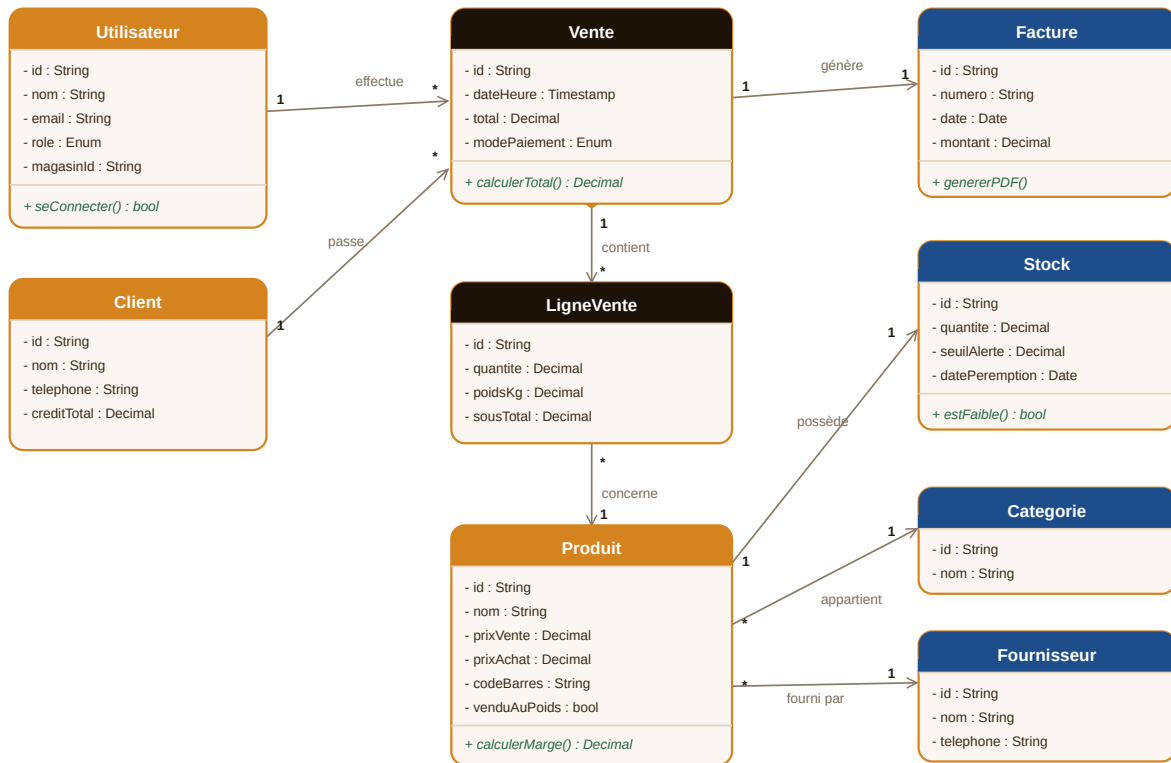


Diagramme de séquence — Processus de vente

Ce diagramme illustre les interactions chronologiques entre le Caissier, l'application mobile, la couche service (Firestore) et l'imprimante lors d'une vente complète.

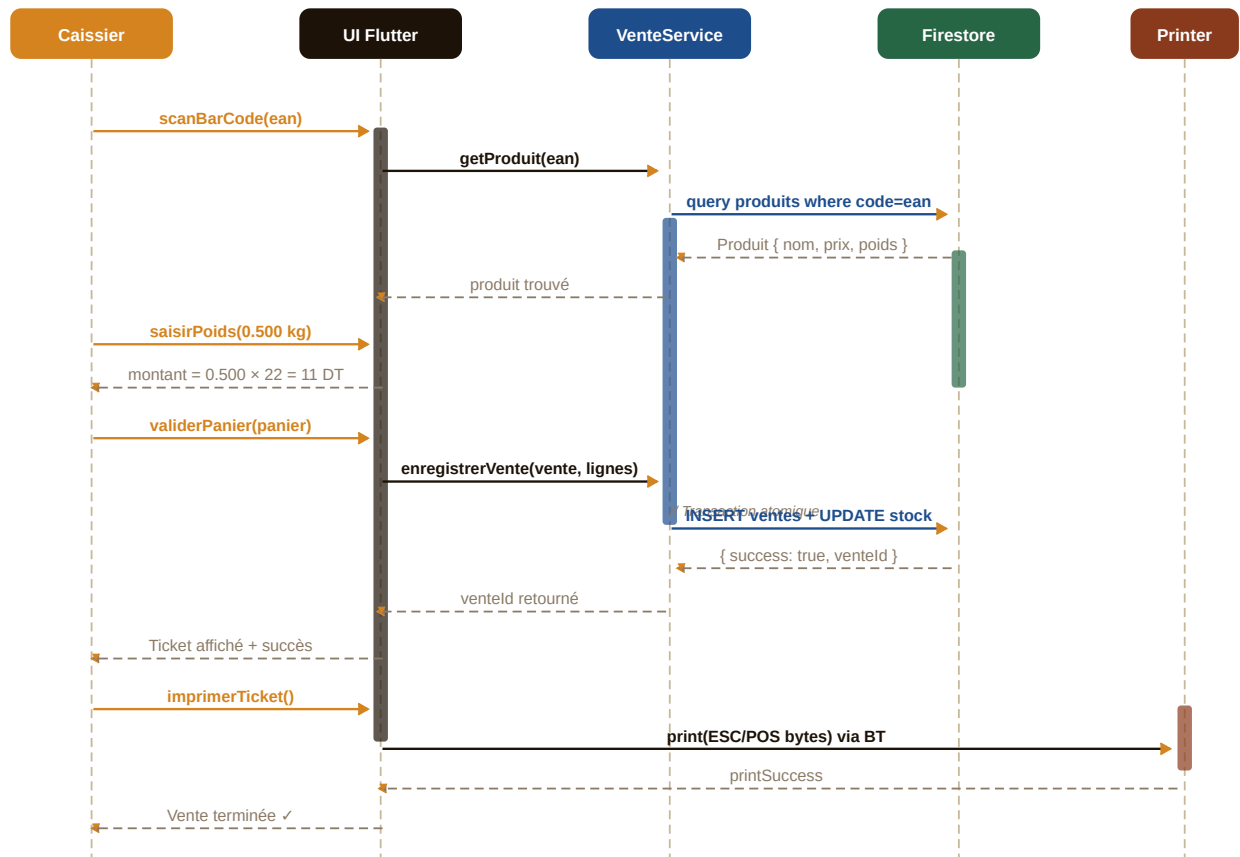
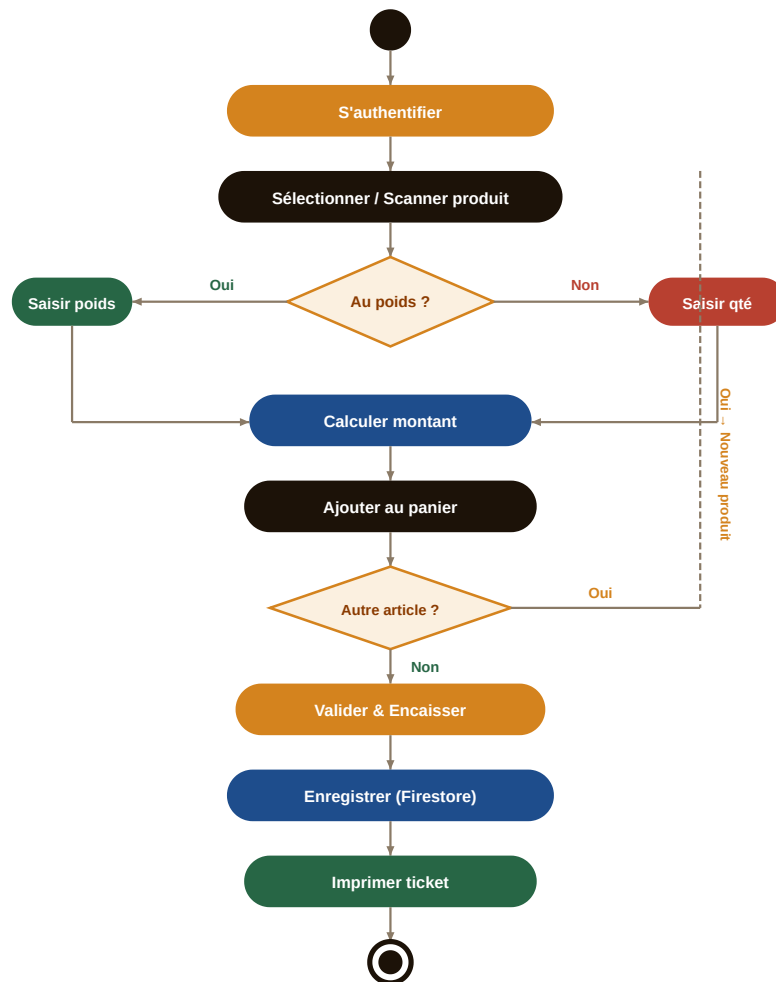


Diagramme d'activités — Processus de vente

Ce diagramme représente le flux d'activités complet pour réaliser une vente, depuis l'ouverture de session jusqu'à la clôture du ticket, en tenant compte des deux modes : vente normale et vente au poids.



Cas de tests fonctionnels

Ces cas de tests valident les fonctionnalités critiques du MVP avant livraison. Chaque test est associé à un scénario nominal et à un scénario d'erreur.

#	Cas de test	Préconditions	Actions	Résultat attendu	Statut
T01	Authentification valide	Compte existant	Saisir email + mdp corrects → Connexion	Redirection vers écran selon rôle (admin/caissier)	À tester
T02	Auth échouée	—	Saisir mot de passe incorrect	Message d'erreur clair, pas de redirection	À tester
T03	Vente au comptoir	Produit en stock (qté > 0)	Scan → Ajouter panier → Valider → Espèces	Vente enregistrée, stock décrémenté, ticket imprimé	À tester
T04	Vente au poids	Produit venduAuPoids = true	Sélect. produit → Saisir 0,500 kg → Ajouter	Montant = prix/kg × 0,5 ; ajout correct au panier	À tester
T05	Alerte stock faible	quantite <= seuilAlerte	Consulter écran Stock	Badge « FAIBLE ⚠ » visible + notification push	À tester
T06	Mode hors-ligne	Connexion Internet coupée	Effectuer une vente	Vente enregistrée localement (SQLite) ; sync auto à la reconnexion	À tester
T07	Accès non autorisé	Utilisateur rôle = caissier	Tenter d'accéder au tableau de bord admin	Accès refusé, message explicite affiché	À tester
T08	Génération de facture	Vente validée (T03)	Ouvrir l'historique → Sélect. vente → Exporter PDF	PDF généré avec numéro, date, articles, total, TVA	À tester
T09	Calcul de marge	prixAchat et prixVente définis	Consulter fiche produit → Afficher marge	marge = prixVente – prixAchat (affiché en DT et %)	À tester
T10	Crédit client	Client existant avec crédit	Vente → Mode paiement = Crédit → Valider	Solde client mis à jour, historique crédit enregistré	À tester

Budget & faisabilité technique

Coûts de développement (stage)

Ressource	Coût estimé
Développeur Flutter (stagiaire)	Stage rémunéré ou PFE
Firebase Spark (gratuit)	0 DT / mois
Figma (maquettes)	Gratuit (plan étudiant)
Google Play Store	≈ 75 DT (one-time)
Apple App Store	≈ 340 DT / an
Domaine + hébergement web	≈ 50 DT / an
Total lancement MVP	≈ 465 DT

Coûts d'exploitation (mensuel)

Service	Plan gratuit	Plan payant
Firebase Firestore	50k lectures/jour	~ \$25/mois
Firebase Auth	10k utilisateurs	\$0.0055/user
Firebase Storage	5 GB	\$0.026/GB
Viabilité	Le plan gratuit couvre les 200 premiers magasins	

Faisabilité technique

✓ Points forts

- Flutter est une technologie mature, largement documentée et avec une grande communauté.
- Firebase élimine le besoin d'un backend custom pour le MVP.
- L'équipe n'a besoin que d'un seul développeur pour couvrir Android et iOS simultanément.
- Le mode offline (SQLite) est natif dans Flutter, sans librairie tierce complexe.

Estimation de rentabilité

10 abonnés Pro	300 DT/mois
50 abonnés Pro	1 500 DT/mois
200 abonnés Pro	6 000 DT/mois
Seuil de rentabilité	~ 20 abonnés

Risques du projet & mitigation

Identification des risques potentiels, évaluation de leur probabilité et de leur impact, et stratégies de mitigation pour chacun.

#	Risque	Prob.	Impact	Stratégie de mitigation
R01	Dépassement des délais — fonctionnalités trop nombreuses pour la durée du stage	Élevée	Élevé	Périmètre MVP strict dès le départ. Fonctionnalités avancées en backlog priorisé.
R02	Problèmes de connexion Firebase — réseau instable en commerce	Moyenne	Élevé	Architecture offline-first avec SQLite. Sync différée et réconciliation automatique.
R03	Adoption utilisateur — résistance au changement des commerçants	Moyenne	Moyen	UX ultra-simple, onboarding guidé, interface bilingue, formation courte (< 30 min).
R04	Sécurité des données — fuite d'informations commerciales sensibles	Faible	Élevé	Firestore Security Rules strictes, chiffrement SQLite, tokens JWT, isolation par storeId.
R05	Concurrence rapide — un concurrent lance une solution similaire	Moyenne	Moyen	Différenciation sur la vente au poids, le prix et les spécificités tunisiennes. Itérations rapides.
R06	Coûts Firebase — dépassement du plan gratuit à la mise à l'échelle	Faible	Moyen	Monitoring des quotas Firebase. Migration vers backend propre (Node.js) si > 200 magasins.
R07	Compatibilité imprimante — diversité des modèles de printers Bluetooth	Moyenne	Faible	Protocole ESC/POS standard supporté par 95% des imprimantes thermiques du marché.

Planning du projet

Découpage en phases sur 8 semaines (2 mois). Les phases peuvent se chevaucher légèrement selon l'avancement. L'objectif est de livrer un MVP fonctionnel et testé à mi-parcours.



SEMAINES 1-2

Phase 1 — Analyse & Conception

Étude du besoin, cahier des charges, maquettes Figma (toutes les vues), schéma de base de données, diagrammes UML. Mise en place du dépôt Git et de l'environnement de développement.



SEMAINES 2-3

Phase 2 — Fondations techniques

Structure du projet Flutter (architecture en couches), configuration Firebase, authentification et gestion des rôles, navigation par profil, modèles de données (Models) et premier écran fonctionnel.



SEMAINES 3-5

Phase 3 — Développement du cœur métier (MVP)

Mode caisse (panier, scan, paiement), vente au poids, gestion des produits (CRUD), gestion de stock avec alertes, génération et impression de factures Bluetooth.



SEMAINES 5-6

Phase 4 — Tableau de bord & Fonctions avancées

Dashboard administrateur (KPIs, graphiques), mode hors-ligne (SQLite + sync), crédit client, calcul de marge, alertes de péremption, clôture de caisse.



SEMAINES 7

Phase 5 — Tests & Optimisation

Exécution des cas de tests fonctionnels, correction des bugs, optimisation des performances (temps de chargement, fluidité), tests sur appareils réels (Android), tests utilisateurs avec un commerçant pilote.



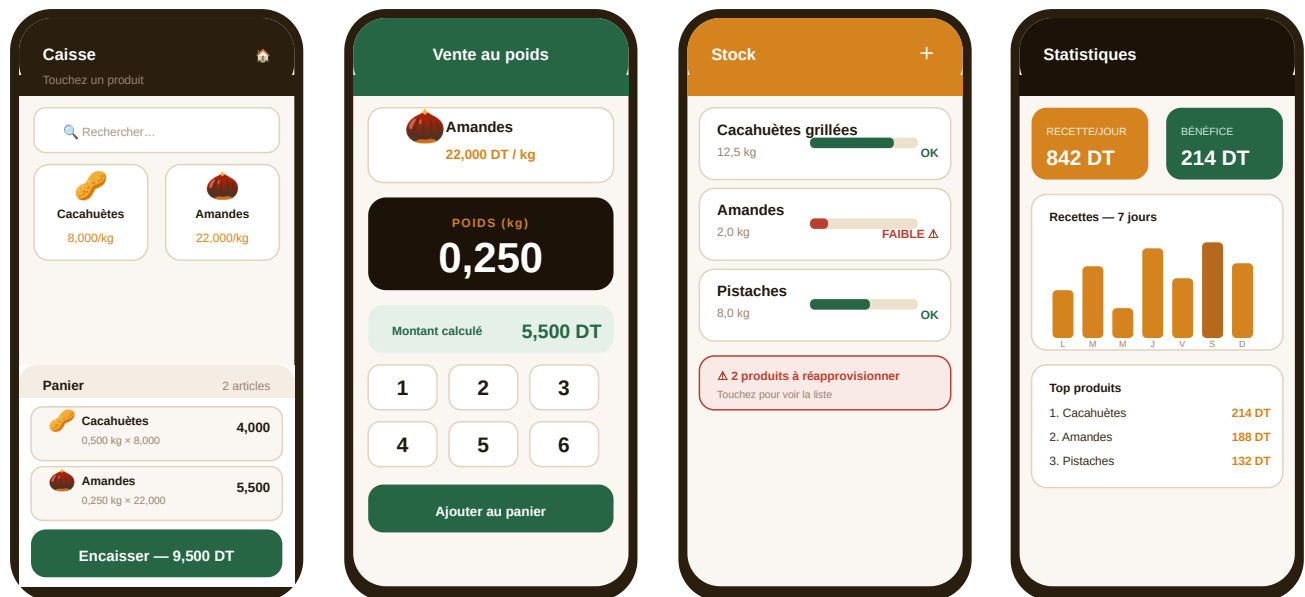
SEMAINE 8

Phase 6 — Documentation & Livraison

Rédaction du rapport de stage, documentation technique (README, guide d'installation), préparation de la démonstration finale, déploiement de la version bêta sur le Play Store.

Maquettes des écrans clés

Maquettes haute fidélité illustrant les parcours principaux. Le design final sera affiné en phase de conception via Figma.



Mode caisse

Vente au poids

Gestion de stock

Tableau de bord

Conclusion & perspectives

Un vrai besoin marché

Des milliers de commerces de proximité en Tunisie cherchent une solution de caisse simple, mobile et abordable. Le segment est sous-servi par les acteurs existants.

Une solution différenciée

Seule solution mobile avec vente au poids native, bilingue, conforme à la réglementation tunisienne et distribuée en modèle SaaS abordable.

Un projet réalisable

Stack technologique moderne et éprouvée (Flutter + Firebase), périmètre MVP maîtrisé, architecture scalable dès la conception.

Pourquoi Sou9i a du potentiel commercial

Sou9i répond à un problème concret et documenté — la sous-digitalisation des commerces de proximité tunisiens — avec une solution moderne, accessible et culturellement adaptée. Le modèle SaaS génère des revenus récurrents prévisibles dès 20 abonnés payants. Le projet est ambitieux mais réaliste : un MVP solide est livrable en 1 à 2 mois de stage, avec une feuille de route claire d'évolutions à forte valeur (intelligence artificielle de réapprovisionnement, intégration des balances connectées, programme de fidélité, expansion géographique).

Je reste disponible pour présenter une démonstration interactive du prototype et approfondir tout aspect technique ou commercial. Ce projet peut être développé en stage ou en PFE et constitue une base solide pour un produit commercialisable.

**Sou9i**

Votre caisse intelligente, dans votre poche. · Flutter + Firebase · Tunisie